

Interactive Online Compiler with Code Visualization and Optimization for Programming Languages

Syed Ali Fathima R
Computer Science and Engineering
Kalasalingam Academy of Research
and Education
Virudhunagar, India
[syedalifathima207@gmail.com](mailto:syedatifathima207@gmail.com)

Abhijeet Kumar
Computer Science and Engineering
Kalasalingam Academy of Research
and Education
Virudhunagar, India
abhijeetkumar1022003@gmail.com

Rishikesh Sourabh
Computer Science and Engineering
Kalasalingam Academy of Research
and Education
Virudhunagar, India
rishikesh.sourabh@gmail.com

Gaurav Kumar
Computer Science and Engineering
Kalasalingam Academy of Research
and Education
Virudhunagar, India
gaurav06@gmail.com

Ayush Kumar Verma
Computer Science and Engineering
Kalasalingam Academy of Research
and Education
Virudhunagar, India
ayushverma.ara19@gmail.com

Abstract— Online programming environments have expanded considerably, but the majority of the web-based compilers available to date only run code and do not give any indication of program behaviour, debugging support, or improvement. This results in a significant learning and productivity gap for students and developers needing a greater understanding of running code and methods for improvement. To address this gap, we present an Interactive Online Compiler that incorporates Code Visualization and Optimization for programming in multiple languages. The proposed system uses Docker-based containerization to execute programs securely and scalable, and WebSockets to transmit output, events, and state to the user in real time. The visualization module provides a clear visualization of control flow, memory states, and variable changes. The proposed solution also provides an optimization engine that performs static and dynamic analysis in runtime and can recommend improvements in efficiency, readability, and maintainability. The proposed interactive online compiler unifies an execution environment, visualization engine, and optimization program within a single, easy-to-use platform, addressing the limitations of traditional online compilers and helping to improve programming educational experiences as well as promoting professional development workflows with increased productivity.

Keywords— Online Compiler, Code Visualization, Code Optimization, Programming Languages, Real Time Execution, Docker, WebSocket, Interactive Programming Compiler.

I. INTRODUCTION

The paper introduces the fully integrated online platform known as the Interactive Online Compiler with Code Visualization and Optimization for Programming Languages [11]. The core thesis demonstrates that combining safe multi-language code execution with real-time visualization and context-based optimization guidance inside a unified browser interface produces enhanced programming understanding for

students along with shortened development cycles for experienced developers [6].

Online coding platforms along with browser-based programming environments now serve as crucial tools for both educational [4]. settings and technical assessment processes and fast development workflows [16]. Users can access compilation and execution services through these platforms without needing complicated local installation processes. Modern program analysis tools combined with debugging visualizers and automated code review tools have developed powerful solutions that enhance both execution flow comprehension and code quality improvement [10]. The capabilities exist in separate ecosystems because debugging and visualization tools limit themselves to particular programming languages while optimization tools operate independently from execution environments and profiling tools work only within desktop IDE environments. Users who work within browser-based environments do not receive a combined real-time overview that connects code with runtime behaviour and actionable guidance [18].

The tools exist separately but researchers have not addressed this issue properly. Most online compilers today perform basic execution only but they lack features for runtime inspection or optimization support [15]. Available visualization tools fail to provide comprehensive performance tracking capabilities along with support for executing multiple programming languages [3]. Optimization systems deliver fixed recommendations yet they fail to use runtime profiling data for delivering educational feedback [4]. A unified platform needs to exist which delivers secure scalable execution together with detailed visualization and smart optimization functions across all programming languages using browser-native technology [10].

The system presented in this paper solves this problem through a modular structure which can be expanded in future developments [17]. Lightweight Docker containers run programs to provide isolation together with reproducibility and multi-language support with strong security guarantees. Real-time execution events pass through a WebSockets-based communication pipeline which delivers input and output alongside call stack activity and variable updates together with performance metrics directly to the browser. The visualization

module converts these events into user-friendly graphical representations which include control-flow graphs and timeline-based execution traces and interactive memory state inspectors [12]. The optimization engine uses both static analysis and dynamic profiling to create recommendations which enhance code efficiency and readability and maintainability. These components work together to change code execution into an active learning and productivity tool [14].

The impetus for this project originated from pedagogical and pragmatic perspectives. For learners, directly visualizing runtime behavior lessens cognitive overload, clarifies the abstract notions of programming, and avoids misunderstandings common among learners [14]. For developers, having actionable optimization advice decreased the cycle time between profiling and optimizing, and containerized execution guarantees a safe and repeatable environment for shared coding and integration pipelines. By integrating execution, visualization, and optimization into one system, we tackle online compilers anew, transforming them from a mere compiler into a platform for collective, insight-driven programming for both education and professional programming [5].

According to research, programming students must learn basic principles about computing systems while they are learning fundamental knowledge of coding to improve their problem-solving and design skills for systems. The ability to visualize the execution of programs is critically important to cs education since it promotes developing essential capabilities for students understanding the structure and operational behavior of programs. Students who have capabilities for visualization learn programming [4].

II. LITERATURE REVIEW

Yang introduced VegaPlus, a system that optimizes interactive data visualizations through combining query rewriting, cost modeling, and machine learning-based optimization in the visualization pipeline. Unlike most traditional visualization languages like D3, Plotly, and Vega, which are concerned with rendering without query optimization, VegaPlus fills the gap by rewriting dataflow operations into SQL queries that run on DBMSs like PostgreSQL and DuckDB. The system opportunistically responds to user interactions, utilizing plan selection heuristics and caching strategies to maximize scalability and responsiveness in real-time applications [1].

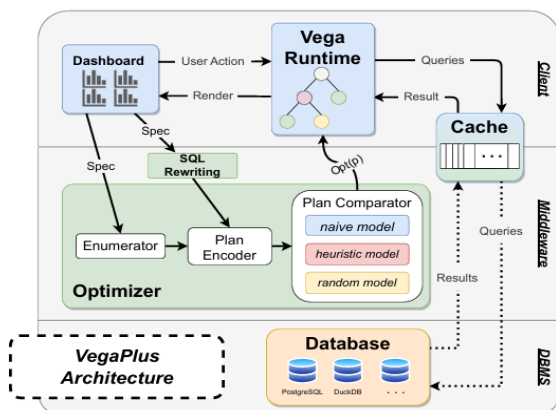


Fig. 1. The architecture diagram of Code Visualizer.

The Fig 1. illustrates VegaPlus Architecture, where a dashboard interface is linked to databases through an optimizer and runtime environment [1].

Mullanu, Petchoo, and Chua created a Code Complexity Analyser and Visualiser in order to assist novice programmers to learn better coding habits through understanding software complexity with machine learning and visualization [2]. The researchers recognized the importance of CCN, and code characteristics, such as lines of code, parameters, and control structures, when measuring code complexity [2]. Six machine learning models for estimating code complexity were designed, trained, and evaluated on a dataset of JavaScript programs, with CART and KNN being the two most successful models for estimating complexity. To further bridge the usability of their findings, the researchers developed a web-based prototype visualising distributions of metrics such as CCN, lines of code, conditionals, and levels of nesting, as a mechanism for the students to begin discussing things in code they struggled with their research demonstrates the merits of comparing formal analysis of complexity with a visual medium for instruction in how novice programmers come to understand programming behaviours and when and how to re-factor complexity in code.

Vrachnos and Jimoyiannis [3]. came up with DAVE, a web-based dynamic algorithm visualization environment aimed to enable novice learners to comprehend simple algorithms, especially array operations. Unlike other tools like Animal, Jeliot, and Trakla2 that needed to be installed or were not interactive, DAVE was developed using HTML5 and JavaScript for immediate execution in a browser with real-time visual feedback as students edited code and entered input data [3]. Their trial with secondary school students revealed that the tool enhanced understanding, promoted experimentation, and facilitated the identification of logical errors and constituted an effective method for learning algorithmic thinking.

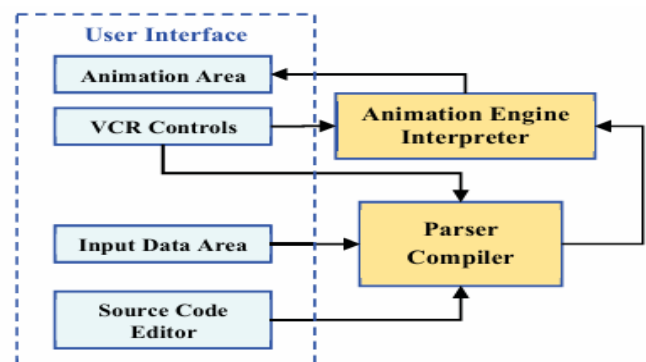


Fig. 2. The architecture diagram of Code Optimizer.

Fig 2 shows the architecture of a system that consists of input data, controls, a code editor, and a user interface with animation, that communicates with the interpreter of an animation engine to visualize the input [3].

Ghandge created AlgoAssist - a combined algorithm visualization and coding environment to enable remote classroom learning. The system consists of a variety of functionalities including pre-assessment, algorithm explanation, line-by-line visualization, coding playground, assignments, and post-assessment within a shared

environment [4]. Unlike most visualizers that are experience limited to the presentation of algorithms, AlgoAssist has features of teacher-student interaction, manual and automated evaluation, performance tracking, and collaborative discussion boards. The platform uses ReactJS as a front end, Django as the back end, and MongoDB to store data with an option of compiler integration for different programming languages. Major functions such as code tracing, leader boards, and anti-cheating, made it especially fitting for online engagement, testing and collaboration with students during the COVID-19 pandemic. The research showed that AlgoAssist not only enables visualization to support improved algorithm understanding- in addition engagement evaluation and collaboration through the online classroom community [4].

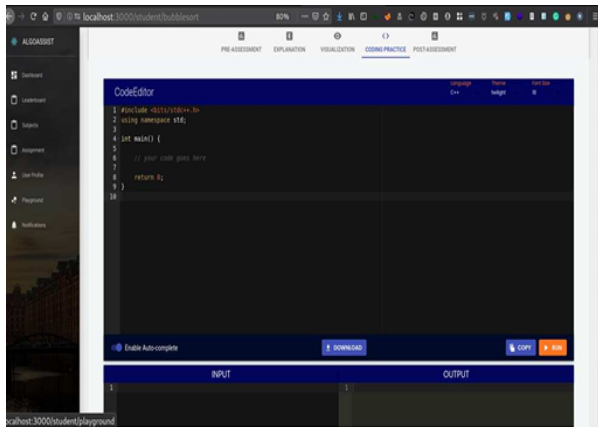


Fig. 3. Coding environment of AlgoAssist.

Fig 3 illustrates the overall system that provides end-to-end animation within a standard DSA [4].

An Online Compiler and Code Execution Platform was discussed by Sharma et al [5] that enables users to write, compile, and execute code instantly, using a web browser, without requiring a local installation of a compiler. It has been implemented for several programming languages while providing developers and students with an intuitive and easy-to-use interface. Significant characteristics of this platform are its accessibility and independence of platform, which allow the end user to execute a program from any connected device however, the principal focus of this platform has been on compiling and executing code which restricted any advanced features that would enable visualizations or modified capabilities [5].

Joose Saino, Alexandre Mercat, and Jarno Vanne, "Live Demonstration: VVC Intra-Coding Visualizer" entails a visualizer with interactive capabilities which displays the encoding process. The VVC open-source intra encoder uvg266 operates under the hood [6]. The visualizer allows real-time identification of block structures, block modes, transforms, and parallelization approaches while VVC is encoding. The visualizer has a user-friendly interface for users to dynamically change encoding tools and parameters. For these reasons, the visualizer is useful for researchers, developers, and students to learn the VVC coding process, tune performance, and assist in educational purposes [6].

Shah and Guyer [7]. created an "interactive call graph visualization tool" to enhance comprehension of programs in large codebases. Classic node-link graphs get entangled with

thousands of functions, so the authors suggested a grid-based microarray layout that shows all functions densely but permits selective examination of call relations. The tool supports filtering, call-tree extraction, and animation, and integrates LLVM and Soot for C++ and Java programs [13]. Case studies demonstrated it assists developers to find important functions, examine performance bottlenecks, and comprehend concurrency problems, and hence it is a valuable contribution to software visualization.

"Code Confabulator: Using LLMs to Generate Code to Visualize" presents an architecture that uses Large Language Models (LLMs) rather than conventional compilers to produce data for code visualization in such languages as Python, Java, C, and C++. The system solves some of the shortcomings of current tools by providing a human-like comprehension of code [8]. LLMs accept freeform inputs, produce structured outputs, and these are mapped to visualizations through a rule-based methodology with the objective of enhancing visualization quality, minimizing complexity, and maximizing adaptability for learning and debugging reasons.

"NUVER: Network Based Vulnerability Visualizer" aims to address the growing need for a strong cybersecurity vulnerability analysis based on a validated two-phase model grounded in time-stamped vulnerability disclosures. In phase one the system produces a text network that uses Natural Language Processing (NLP) techniques to characterize vulnerabilities and their relationship to each other [9]. In phase two the system provides an analysis of the network theoretic framework of the vulnerabilities based on complex interactions between vulnerabilities. A network theoretic method affords security researchers the ability to think about the relationships between vulnerabilities and how they change over time.

"Unimapper: An Online Interactive Analyzer or Visualizer of Optical Mapping Experimental Data" presents a web-based environment for the exploration and visualization of complex cardiac electrophysiology data. Conventional approaches share data as movies or plots, which are not interactive to explore. Unimapper overcomes this limitation by allowing for online computation and visualization of experimental recordings or simulations in data cubes [10]. It provides functionality for multi-channel visualization, general image and signal processing, and high-end phase calculations based on the Hilbert transform.

The existing online compilers underpinning most of the tools currently focus on compiling and executing code without any built-in visualization or optimization capabilities. Further, visualization tools offer execution capabilities in a limited set of programming languages and fall short of dynamically presenting actual runtime behavior with constant updates to the visualization. In addition, optimization features tend to mainly be based on some static recommendations without the context of runtime. Overall, in terms of a singular feature set that includes secure, containerized execution, a step-wise visualization of code execution, and dynamically updating optimization suggestions, there is no currently available tool that includes these features in one single web-based platform. This check on the literature and previous attempts showcases the nature of the need for this project to provide a fully integrated approach that combines execution with visualization in real-

time, and optimization feedback based in part on the contextual cues of the program and execution.

III. PROPOSED METHODOLOGY

The Interactive Online Compiler with Code Visualization and Optimization operates through a web-based modular framework which integrates secure code execution with live event monitoring and detailed client-side visual output and context-specific optimization recommendations. The platform takes user-submitted source code to compile and execute within secure containerized environments before extracting detailed runtime events and performance metrics that serve both browser visualization and optimization engine input. Users can experience an interactive platform that lets them observe code execution while providing instant access to performance improvement suggestions and code quality recommendations.

The system achieves three fundamental goals at the highest level through its design: execution independence through containerization and security measures and real-time visualization client event delivery with precise ordering and dynamic runtime data analysis for generating contextual optimization feedback.

The current version of the software allows for coding in four programming languages: Java. Each execution Container has specified resource limits: one CPU Core(512MB) of RAM, with a maximum of five seconds of execution time. During run-time execution, execution Trace data will be collected and sent to the visualization and optimization modules. The execution Trace will detail the execution of line-by-line events, variable updates, loop iterations, and function calls (Fig.4).

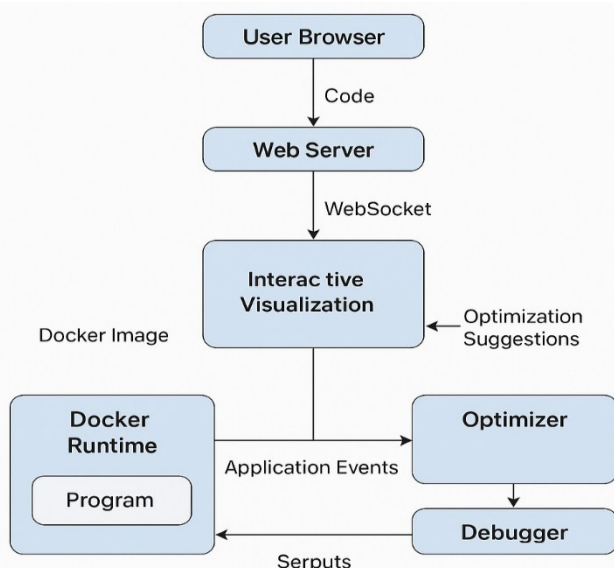


Fig. 4. Block diagram of Proposed Methodology illustrates container optimization, real-time execution.

A. Code Submission and Input Handling

Users start writing code within the Monaco Editor which delivers instant syntax highlighting and error detection together with autocompletion features. The user completes code development before entering input values which will trigger program execution. The API gateway receives both

code and input values which undergo validation before preparing them for backend processing. The system guarantees that only valid and purposeful submissions reach the execution layer.

B. Containerized Execution Environment

The submitted code operates within a containerized environment which utilizes Docker for execution. Each container contains predefined installations of compilers and interpreters that support the programming languages. This design guarantees security, as every execution runs in isolation, and containers are destroyed after use. Strict resource limitations regarding CPU, memory and timeouts are implemented to prevent system overload or malicious usage.

C. Real-Time Communication via WebSockets

During execution, the results of your workflow will flow back to the front-end in real time, via WebSockets. This allows the user to see results as they are produced, both because the system is streaming output in real-time during execution instead of forcing the user to wait for the superior task to finish and because intermediate results & logs recorded at runtime – including error messages – can be streamed live for debugging and interactive development.

D. Code Visualization Layer

The execution trace enters the visualization layer which transforms program behavior into graphical displays. The execution visualization includes flowcharts that display control structures alongside variable state transformations and memory allocation patterns throughout program execution. Users can observe the program flow one step at a time to enhance their understanding of programming logic as well as their ability to identify errors and grasp programming concepts through visual learning.

E. Optimization Engine

The optimization engine examines program code through two distinct analysis methods apart from executing the code:

- Static analysis assesses code before execution to find variables that remain unused and inadequate development methods and intricate code structures.
- The system monitors operational performance and resource usage and operational efficiency through dynamic analysis. The system generates improvement recommendations by analyzing the results which include changing inefficient loops and memory optimization along with selecting superior algorithms.

F. Debugger and Execution Modes

The system provides two execution modes. When running in normal mode the program executes sequentially from beginning to end while delivering output directly to the user. The debug mode enables users to execute programs one line at a time while allowing them to insert breakpoints and examine variable states throughout the program execution. This flexibility helps both beginners and advanced programmers gain deeper insight into their code.

G. Backend Services and Security

The backend performs execution orchestration as well as resource scheduling and container management while responding to user requests. The implementation of sandboxing together with system call restrictions and API authentication provides protection from harmful code.

Through log monitoring and execution tracking the system maintains stability when processing large workloads.

H. Frontend User Experience

Users access an IDE-like interface through the Monaco Editor to perform their frontend tasks. Users can perform code development and debugging directly through their web browser without any requirement for local software installations. The unified interface presents execution results alongside visualizations and debugger controls and optimization hints within a single workspace environment. A professional and engaging user experience results from seamless transitions between code writing and running and analysis tasks.

I. Evaluation

The assessment of this system depends on how fast it executes operations as well as its visualization accuracy and optimization suggestion quality and the quality of its user interface experience. The evaluation was conducted using representative code samples ranging from basic conditional statements and loops to multi-function programs, allowing assessment of execution accuracy, visualization correctness, and optimization relevance.

IV. EVALUATION AND RESULT

The system was assessed using numerous code snippets ranging from basic loops to complex modular programs involving multiple functions. The visualization engine accurately produced execution flows that dynamically displayed function calls and loops and branching conditions for each test case, with no errors.

The performance showed minimal lag time from the initial submission of code to the rendering of visualization output, even with all of the input variables being set very high to test the limits of usage. The system was expected to be a good use of resources with its containerized execution and optimized backend processes, with only minimal delay on both the frontend and the backend. Users similarly reported improved usability by using the interactive debugger, and with multiple modes of execution that made it possible to iterate on the code dynamically without interference to performance or reliability.

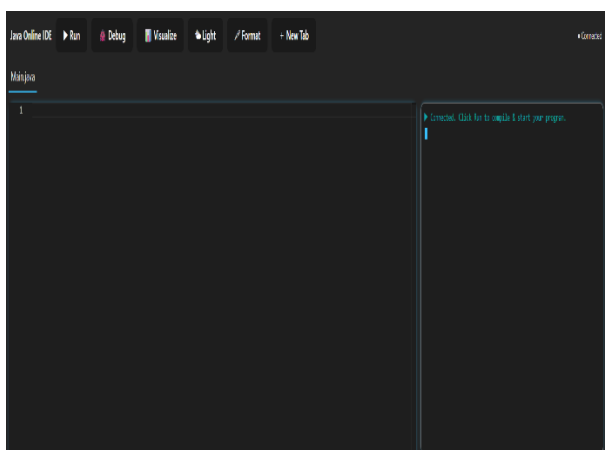


Fig. 5. Interface of the Online Compiler Environment.

Fig 5. shows user interface of the proposed cloud-based compiler environment. The collaborative code editor is displayed in the left-side panel, with the right-side panel displaying our program's live output. The main features of execution, debugging, visualization, and formatting appeared

at the top with the toolbar, allowing an easy and interactive user experience.

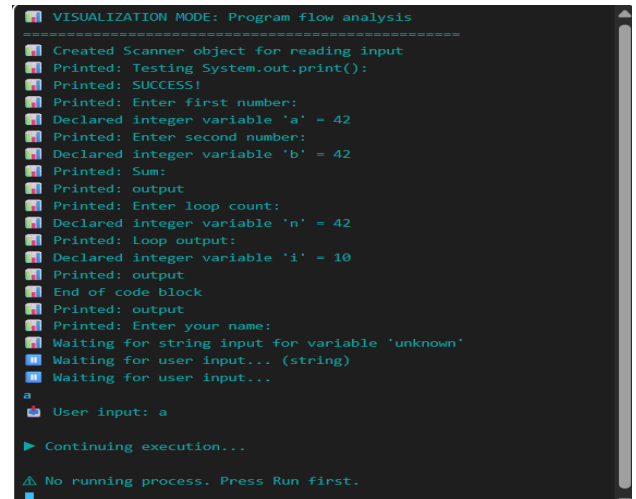


Fig. 6. Visualization of Program Execution Flow.

The system is also shown in its visualization mode in Fig. 6, which traces code execution in real time and step by step - both of which are automated. Every operation, such as the declaration of variables, input manipulation, and loops, is traced and displayed in real time. The tracing system provides the additional context of program behavior to the user and helps illustrate the logic flow and possible runtime errors.

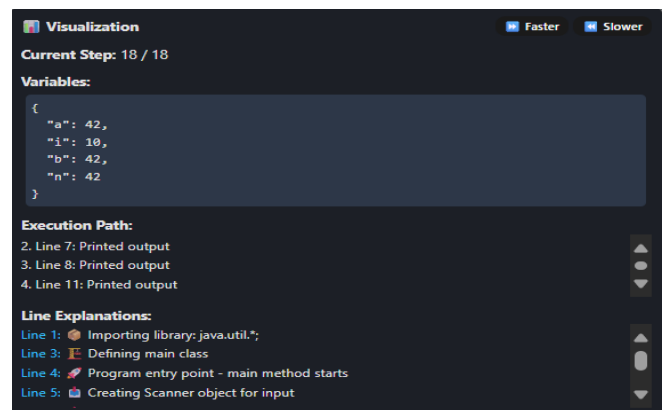


Fig. 7. Line-by-Line Visualization of Code Execution.

Fig. 7 illustrates the system's visualization mode in which execution is described line by line with supporting details. With each step, the tool indicates the current active line, execution path, as well as the current value of variables. This makes it easy for users to see how the program state evolves through execution.

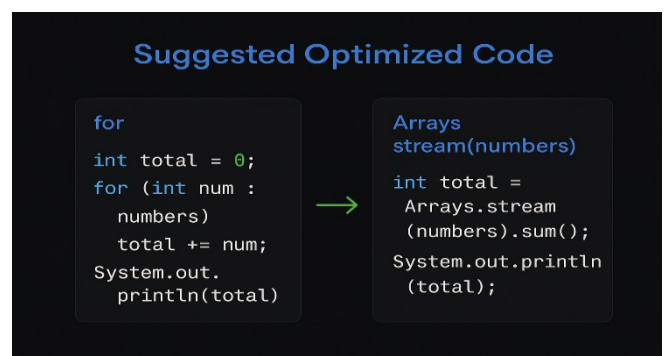


Fig. 8. Code Optimizer.

The Code Optimizer, illustrated in Fig. 8, improves the performance of a program by eliminating redundant dimensions. It analyzes the logic and suggests a method, that is shorter or faster, to reach the same result. In the example shown, a loop that was anticipated to be done by hand is now expressed in a native method. This results in a smaller code syntax while obtaining the same result. Generally, optimizers, by this action, seek to improve visual appeal, enhance efficiency, and limit error possibilities.

In comparison with the existing online compilers, the new system provides improved visibility in terms of program execution by way of real-time tracing and flow visualization using graphical means - all of which are unavailable in web-based execution environments such as existing compilers. The proposed system differs from stand-alone visualizers, such as DAVE, AlgoAssist, or CodeLens, by integrating execution and visualization together rather than relying on pre-defined algorithm animation. In addition, unlike traditional code optimization tools based on static analysis, the proposed optimization engine uses runtime data for context-aware and relevant suggestions. These comparisons show that the proposed framework integrates functionally at a higher degree and provides more overall functionality for those learning as well as other developers.

V. CONCLUSION

An interactive online compiler with live code visualization has been created in this report, which assists learners in tracking program execution in real-time as well as provides a visual representation of how control flows through a program along with the state of all variables and any intermediate results. The system combines lightweight instrumentation with real-time visualization and contextually optimized toolsets to significantly reduce cognitive overload, increase understanding of programming concepts and significantly improve the speed at which a learner or developer can correct an error by debugging their program. The containerized execution environment enhances both security and scalability. The ability to provide real-time feedback is another major benefit of this system. Currently, the prototype is limited to a relatively small number of programming languages and could have limitations on its scalability when dealing with large, complex or concurrent applications. Future work will include broadening the range of supported languages, enhancing the systems' capabilities for working with larger or more complex concurrent systems, including intelligent optimization using AI-analysis and performing large-scale user studies for measuring the success of the learning and skill acquisition process. Future extensions of the current prototype will include exploring options for deployment in both edge and cloud-based environments, developing better security mechanisms, adding the capability of being integrated into a CI/CD pipeline, and using a variety of flexible visualization methods to enhance the usability of the system for academic and professional programmers.

REFERENCES

- [1] J. Yang, H.K. Joo, S. Yerramreddy, D. Moritz, and L. Battle, "Optimizing Dataflow Systems for Scalable Interactive Visualization," *Proc. ACM Mgmt. of Data*, vol. 2, no. 1, Art. no. 1–25, Mar. 2024, doi: 10.1145/3639276.
- [2] S. Mullanu, S. Petchoo, and C. Chua, "Code Complexity Analyser and Visualiser for Novice Programmer," in *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 2020, doi:10.1109/CSDE50874.2020.9411562.
- [3] E. Vrachnos and A. Jimoyiannis, "DAVE: A Dynamic Algorithm Visualization Environment for Novice Learners" in *Proc. 8th IEEE Int. Conf. Advanced Learning Technologies (ICALT 2008)*, Santander, Spain, July 1–5, 2008, pp. 319–323, doi: 10.1109/ICALT.2008.148.
- [4] A. B. Ghangde, B. P. Udhane, H. R. Yadav, P. S. Thakre, V. G. Kottawar, and P. B. Deshmukh, "AlgoAssist: Algorithm Visualizer and Coding Platform for Remote Classroom Learning" *2021 5th International Conference on Computer, Communication and Signal Processing (ICCCSP)*, 2021, pp. 160–164, doi: 10.1109/ICCCSP52374.2021.9465503.
- [5] A. Sharma, A. Dixit, and B. Upadhyay, "Online Compiler" *International Research Journal of Engineering and Technology*, vol. 4, no. 5, May 2017, pp. 239.
- [6] J. Sainio, A. Mercat, and J. Vanne, "Live Demonstration: VVC Intra-Coding Visualizer," in *Proc. 2025 IEEE Int. Symp. Circuits and systems*, London, U.K., May 25–28, 2025, doi: 10.1109/ISCAS56072.2025.11043749.
- [7] M. D. Shah and S.Z. Guyer, "An Interactive Microarray Call-Graph Visualization," in *Proc. 2016 IEEE Working Conference on Software Visualization*, Eindhoven, Netherlands, Oct. 2016, pp. 86–90, doi: 10.1109/VISOFT.2016.14.
- [8] R. Nannapaneni, P. Pranavi, E. Suryaa, V. K. R. Challa, and M. Belwal, "Code Confabulator: Harnessing LLMs to Compile Code for Visualization," in *Proc. 2024 15th Int. Conf. computing Communication and Networking Technologies*, Kamand, India, Jun. 24 – 28, 2024, doi: 10.1109/ICCCNT61001.2024.10724543.
- [9] H. sparks and K. Ghosh, "NUVER: Network Based Vulnerability Visualizer," in *Proc. 2023 IEEE 30th Annual Software Technology Conference (STC)*, MD, USA, Sep. 25–27, 2023, doi: 10.1109/STCS58598.2023.00010.
- [10] S. Iravanian, I. Uzelac, D. I. Cairns, E. M. Cherry, A. Kaboudian, and F. H. Fenton, "Unimapper: An Online Interactive Analyzer/Visualizer of Optical Mapping Experimental Data," in *Proc. 2021 Computing in Cardiology*, Brno, Czech Republic, Sep. 13 – 15, 2021, doi: 10.23919/CinC53138.2021.9662942.
- [11] A. Trivedi, K. Pandey, V. Gupta, and M. K. Jha, "AlgoRhythm - A Sorting and Path-finding Visualizer Tool to Improve Existing Algorithms Teaching Methodologies," in *Proc. 2023 13th Int. Conf. Cloud Computing, Data Science & Engineering*, Noida, India, Jan. 19–20, 2023, doi: 10.1109/Confluence56041.2023.10048793.
- [12] Y. Guo, S. Bettaieb, Q. Hu, Y. Le Traon, and Q. Tang, "CodeLens: An Interactive Tool for Visualizing Code Representations," *arXiv preprint arXiv:2307.14902*, Jul. 2023, doi: 10.48550/arXiv.2307.1490.
- [13] L. Singh, S. Khare, A. Parvez, and S. Verma, "Research Paper on Path-finding Algorithm Visualizer," in *Proc. 2022 Int. Conf. Cyber Resilience (ICCR)*, Dubai, United Arab Emirates, Oct. 6–7, 2022, doi: 10.1109/ICCR56254.2022.9995925.
- [14] H. W. Robbins, S. C. Gutekunst, D. B. Shmoys, and D. P. Williamson, "GILP: An Interactive Tool for Visualizing the Simplex Algorithm" *arXiv preprint arXiv:2210.15655*, Dec. 2022, doi: 10.48550/arXiv.2210.15655.
- [15] K. Musumbu, "Algorithms Visualization Tool for Students and Lectures in Computer Science," *arXiv preprint arXiv:1403.4423*, Mar. 2014, doi: 10.48550/arXiv.1403.4423.
- [16] N. Parsly, J. Wang, N. West, Q. Zhang, and F. Li, "DiME and AGVis: A Distributed Messaging Environment and Geographical Visualizer for Large - Scale Power System Simulation," in *Proc. 2023 North American Power Symposium*, Asheville, NC, USA, Oct. 15–17, 2023, doi: 10.1109/NAPS58826.2023.10318583.
- [17] R.F. Toledo, J.M. Atlee, R.M. Xiong, and M. Liu, "Neo4j Browser: Visualizing Variable-Aware Analysis Results," in *Proc. 2024 IEEE/ACM 46th Int. Conf. Software Engineering: Companion Proceedings (ICSE-Companion)*, Lisbon, Portugal, Apr. 14–20, 2024, doi: 10.1145/3639478.3640046.
- [18] J. Fan and J. Zhang, "Neural Belief Propagation Decoders for Concatenated Calderbank – Shor – Steane Codes" in *Proc. ICC 2024 - IEEE Int. Conf. Communications (ICC)*, Denver, CO, USA, Jun. 9–13, 2024, doi: 10.1109/ICC51166.2024.10622478.